

Chapter 6

1. a

Best subset selection has the smallest training RSS because the other two methods determine models with a path dependency on which predictors they pick first as they iterate to the k 'th model.

b (*)

Best subset selection may have the smallest test RSS because it considers more models than the other methods. However, the other models might have better luck picking a model that fits the test data better.

c

i. True. ii. True. iii. False. iv. False. v. False.

3.

a

(iv) Steadily decreases: As we increase s from 0, all β 's increase from 0 to their least square estimate values. Training error for 0 β 's is the maximum and it steadily decreases to the Ordinary Least Square RSS

b

(ii) Decrease initially, and then eventually start increasing in a U shape: When $s = 0$, all β 's are 0, the model is extremely simple and has a high test RSS. As we increase s , β 's assume non-zero values and model starts fitting well on test data and so test RSS decreases. Eventually, as β 's approach their full blown OLS values, they start overfitting to the training data, increasing test RSS.

c

(iii) Steadily increase: When $s = 0$, the model effectively predicts a constant and has almost no variance. As we increase s , the models includes more β 's and their values start increasing. At this point, the values of β 's become highly dependent on training data, thus increasing the variance.

d

(iv) Steadily decrease: When $s = 0$, the model effectively predicts a constant and hence the prediction is far from actual value. Thus bias is high. As s increases, more β 's become non-zero and thus the model continues to fit training data better. And thus, bias decreases.

e

(v) Remains constant: By definition, irreducible error is model independent and hence irrespective of the choice of s , remains constant.

4.

a

(iii) Steadily increase: As we increase λ from 0, all β 's decrease from their least square estimate values to 0. Training error for full-blown-OLS β s is the minimum and it steadily increases as β s are reduced to 0.

b

(ii) Decrease initially, and then eventually start increasing in a U shape: When $\lambda = 0$, all β s have their least square estimate values. In this case, the model tries to fit hard to training data and hence test RSS is high. As we increase λ , *beta* s start reducing to zero and some of the overfitting is reduced. Thus, test RSS initially decreases. Eventually, as *beta* s approach 0, the model becomes too simple and test RSS increases.

c

(iv) Steadily decreases: When $\lambda = 0$, the β s have their least square estimate values. The actual estimates heavily depend on the training data and hence variance is high. As we increase λ , β s start decreasing and model becomes simpler. In the limiting case of λ approaching infinity, all *beta* s reduce to zero and model predicts a constant and has no variance.

d

(iii) Steadily increases: When $\lambda = 0$, β s have their least-square estimate values and hence have the least bias. As λ increases, β s start reducing towards zero, the model fits less accurately to training data and hence bias increases. In the limiting case of λ approaching infinity, the model predicts a constant and hence bias is maximum.

e

(v) Remains constant: By definition, irreducible error is model independent and hence irrespective of the choice of λ , remains constant.

10.

a

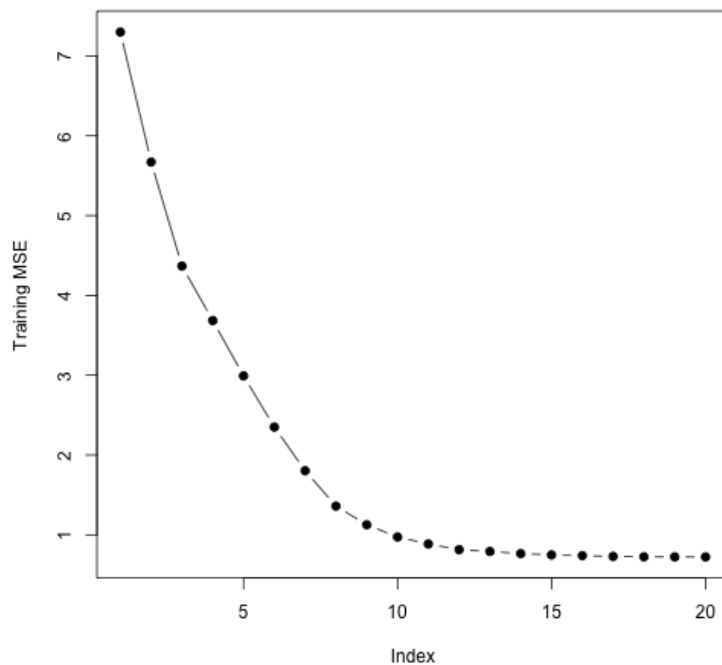
```
set.seed(1)
p = 20
n = 1000
x = matrix(rnorm(n * p), n, p)
B = rnorm(p)
B[3] = 0
B[4] = 0
B[9] = 0
B[19] = 0
B[10] = 0
eps = rnorm(p)
y = x %*% B + eps
```

b

```
train = sample(seq(1000), 100, replace = FALSE)
y.train = y[train, ]
y.test = y[-train, ]
x.train = x[train, ]
x.test = x[-train, ]
```

c

```
library(leaps)
regfit.full = regsubsets(y ~ ., data = data.frame(x = x.train, y = y.train),
  nvmax = p)
val.errors = rep(NA, p)
x_cols = colnames(x, do.NULL = FALSE, prefix = "x.")
for (i in 1:p) {
  coefi = coef(regfit.full, id = i)
  pred = as.matrix(x.train[, x_cols %in% names(coefi)]) %*%
  coefi[names(coefi) %in%
    x_cols]
  val.errors[i] = mean((y.train - pred)^2)
}
plot(val.errors, ylab = "Training MSE", pch = 19, type = "b")
```



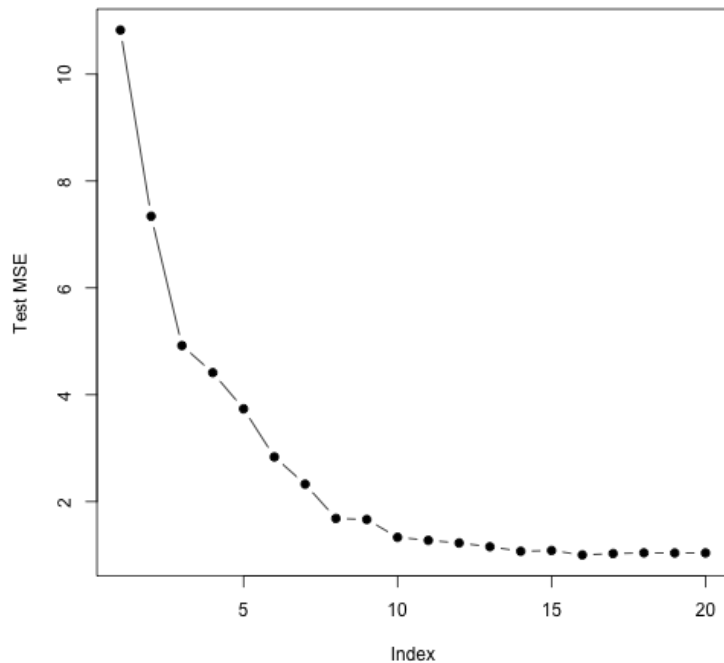
d

```
val.errors = rep(NA, p)
```

```

for (i in 1:p) {
  coefi = coef(regfit.full, id = i)
  pred = as.matrix(x.test[, x_cols %in% names(coefi)]) %*%
  coefi[names(coefi) %in%
    x_cols]
  val.errors[i] = mean((y.test - pred)^2)
}
plot(val.errors, ylab = "Test MSE", pch = 19, type = "b")

```



e

```

which.min(val.errors)
## [1] 16

```

16 parameter model has the smallest test MSE.

f

```

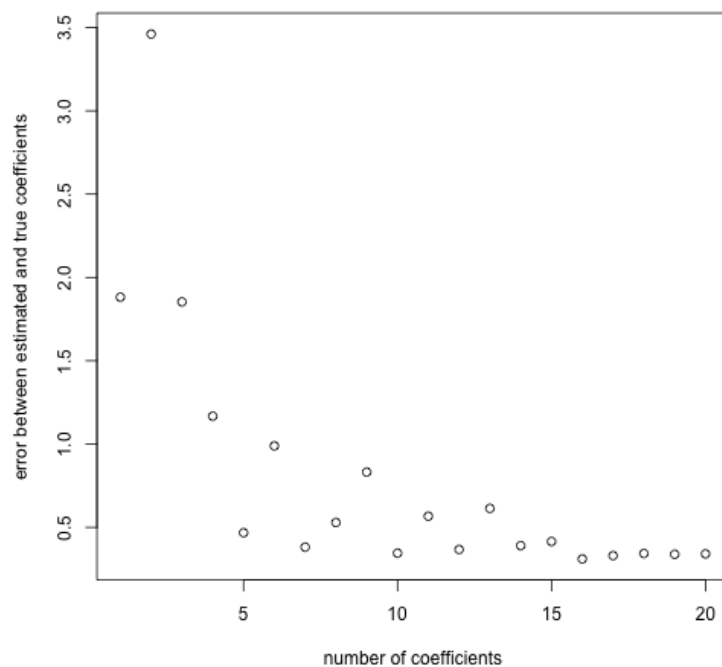
coef(regfit.full, id = 16)
## (Intercept)      x.1      x.2      x.5      x.6      x.7
##   0.09613    0.28257    0.19386    0.99995   -0.28598   -1.50482
##      x.8      x.11      x.12      x.13      x.14      x.15
##   0.77817    0.90816    0.48478   -0.19747   -0.71979   -0.74282
##      x.16      x.17      x.18      x.19      x.20
##  -0.33901    0.12235    1.74270   -0.12435   -1.03003

```

Caught all but one zeroed out coefficient at x.19.

g

```
val.errors = rep(NA, p)
a = rep(NA, p)
b = rep(NA, p)
for (i in 1:p) {
  coefi = coef(regfit.full, id = i)
  a[i] = length(coefi) - 1
  b[i] = sqrt(sum((B[x_cols %in% names(coefi)] - coefi[names(coefi) %in%
x_cols])^2) +
  sum(B[!(x_cols %in% names(coefi))])^2)
}
plot(x = a, y = b, xlab = "number of coefficients", ylab = "error between
estimated and true coefficients")
```



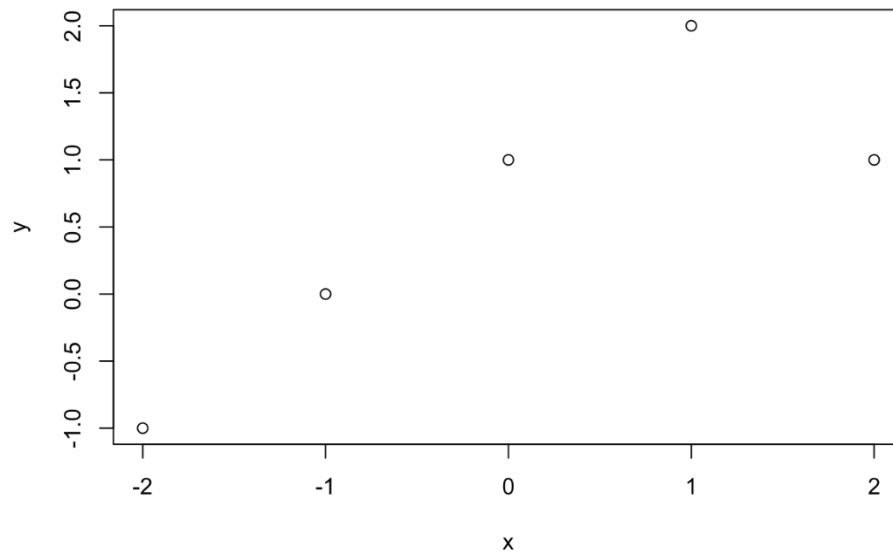
```
which.min(b)
## [1] 16
```

Model with 9 coefficients (10 with intercept) minimizes the error between the estimated and true coefficients. Test error is minimized with 16 parameter model. A better fit of true coefficients as measured here doesn't mean the model will have a lower test MSE.

Chapter 7

3.

```
x = -2:2
y = 1 + x + -2 * (x-1)^2 * I(x>1)
plot(x, y)
```

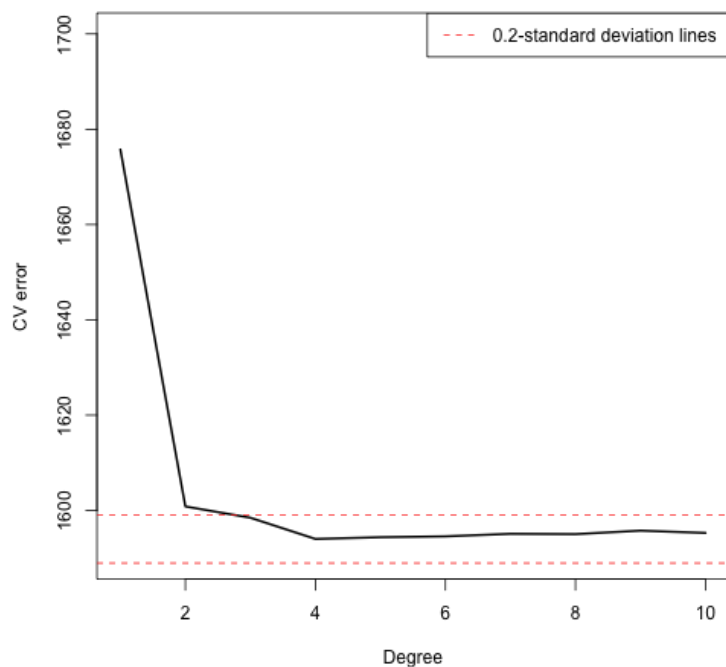


6.

a

Load Wage dataset. Keep an array of all cross-validation errors. We are performing K-fold cross validation with $K=10$.

```
set.seed(1)
library(ISLR)
library(boot)
all.deltas = rep(NA, 10)
for (i in 1:10) {
  glm.fit = glm(wage~poly(age, i), data=Wage)
  all.deltas[i] = cv.glm(Wage, glm.fit, K=10)$delta[2]
}
plot(1:10, all.deltas, xlab="Degree", ylab="CV error", type="l", pch=20,
     lwd=2, ylim=c(1590, 1700))
min.point = min(all.deltas)
sd.points = sd(all.deltas)
abline(h=min.point + 0.2 * sd.points, col="red", lty="dashed")
abline(h=min.point - 0.2 * sd.points, col="red", lty="dashed")
legend("topright", "0.2-standard deviation lines", lty="dashed", col="red")
```



The cv-plot with standard deviation lines show that $d=3$ is the smallest degree giving reasonably small cross-validation error.

We now find best degree using Anova.

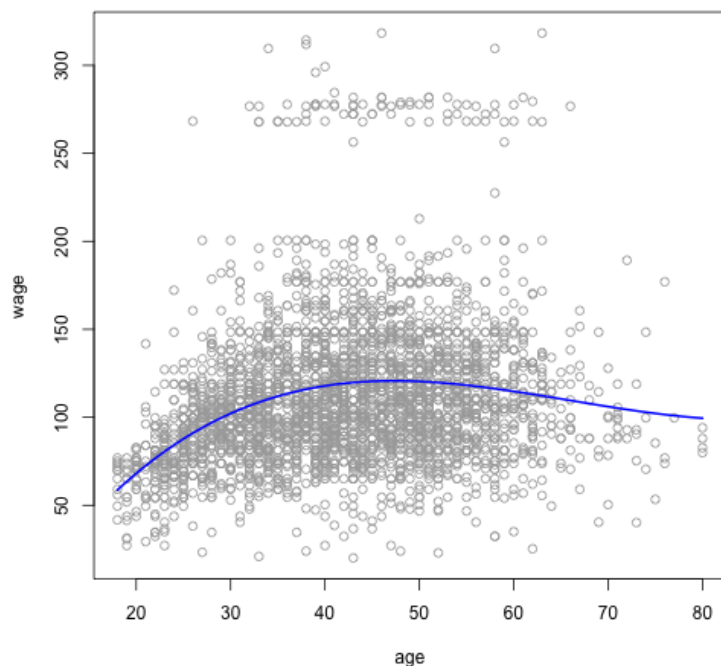
```
fit.1 = lm(wage~poly(age, 1), data=Wage)
fit.2 = lm(wage~poly(age, 2), data=Wage)
fit.3 = lm(wage~poly(age, 3), data=Wage)
fit.4 = lm(wage~poly(age, 4), data=Wage)
fit.5 = lm(wage~poly(age, 5), data=Wage)
fit.6 = lm(wage~poly(age, 6), data=Wage)
fit.7 = lm(wage~poly(age, 7), data=Wage)
fit.8 = lm(wage~poly(age, 8), data=Wage)
fit.9 = lm(wage~poly(age, 9), data=Wage)
fit.10 = lm(wage~poly(age, 10), data=Wage)
anova(fit.1, fit.2, fit.3, fit.4, fit.5, fit.6, fit.7, fit.8, fit.9, fit.10)
## Analysis of Variance Table
##
## Model 1: wage ~ poly(age, 1)
## Model 2: wage ~ poly(age, 2)
## Model 3: wage ~ poly(age, 3)
## Model 4: wage ~ poly(age, 4)
## Model 5: wage ~ poly(age, 5)
## Model 6: wage ~ poly(age, 6)
## Model 7: wage ~ poly(age, 7)
## Model 8: wage ~ poly(age, 8)
## Model 9: wage ~ poly(age, 9)
## Model 10: wage ~ poly(age, 10)
##      Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      2998 5022216
```

```
## 2      2997 4793430 1      228786 143.76 <2e-16 ***
## 3      2996 4777674 1      15756   9.90 0.0017 **
## 4      2995 4771604 1       6070   3.81 0.0509 .
## 5      2994 4770322 1       1283   0.81 0.3694
## 6      2993 4766389 1       3932   2.47 0.1161
## 7      2992 4763834 1       2555   1.61 0.2052
## 8      2991 4763707 1        127   0.08 0.7779
## 9      2990 4756703 1       7004   4.40 0.0360 *
## 10     2989 4756701 1         3    0.00 0.9675
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Anova shows that all polynomials above degree 33 are insignificant at 11 significance level.

We now plot the polynomial prediction on the data

```
plot(wage~age, data=Wage, col="darkgrey")
agelims = range(Wage$age)
age.grid = seq(from=agelims[1], to=agelims[2])
lm.fit = lm(wage~poly(age, 3), data=Wage)
lm.pred = predict(lm.fit, data.frame(age=age.grid))
lines(age.grid, lm.pred, col="blue", lwd=2)
```



b

We use cut points of up to 10.

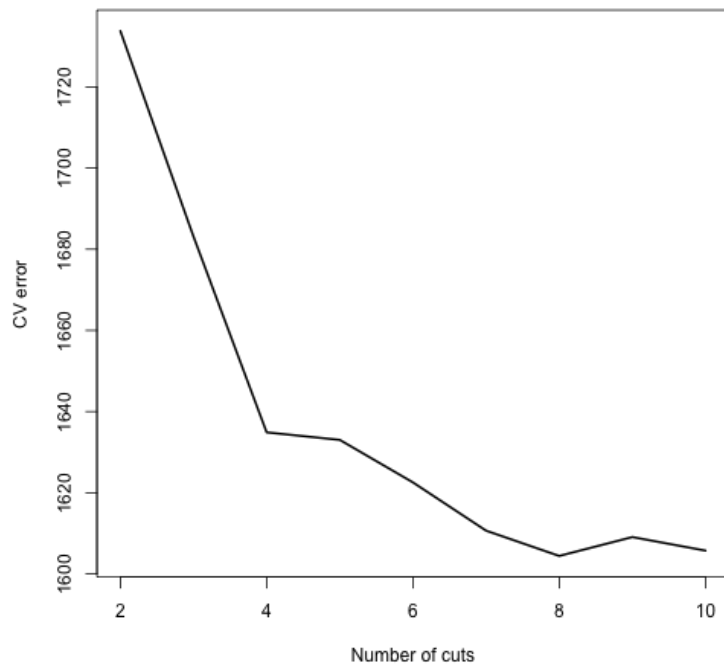
```
all.cvs = rep(NA, 10)
```



```

for (i in 2:10) {
  Wage$age.cut = cut(Wage$age, i)
  lm.fit = glm(wage~age.cut, data=Wage)
  all.cvs[i] = cv.glm(Wage, lm.fit, K=10)$delta[2]
}
plot(2:10, all.cvs[-1], xlab="Number of cuts", ylab="CV error", type="l",
pch=20, lwd=2)

```



The cross validation shows that test error is minimum for $k=8$ cuts.

We now train the entire data with step function using 8 cuts and plot it.

```

lm.fit = glm(wage~cut(age, 8), data=Wage)
agelims = range(Wage$age)
age.grid = seq(from=agelims[1], to=agelims[2])
lm.pred = predict(lm.fit, data.frame(age=age.grid))
plot(wage~age, data=Wage, col="darkgrey")
lines(age.grid, lm.pred, col="red", lwd=2)

```

